

AI-Code Security Platform

Project Proposal

Vidhi Saxena (1024030700)

Yashti Mittal (1024030701)

Rajat Verma (1024030696)

1 Higher-order Goal

More and more code is now written with the help of AI tools. This is fast, but it skips a lot of basic safety habits: a hardcoded password, no check on who can access a page, an old library with a known hole in it.

This project builds one tool that catches these mistakes at every point they can happen: before the code is generated, while it is being typed, when it is pushed to GitHub, and later during a deeper scheduled check.

The bigger goal is to make AI-written code as safe as code written carefully by hand, without slowing developers down.

2 Time-to-value

- We build the smallest useful piece first: one working check, end to end, and get it running quickly, instead of designing everything before writing any code.
- We work in short cycles (weekly), test each new check on real sample repos, and fix what does not work before adding the next feature.
- Success in the first round is simple: one check (say, finding a leaked password) actually catches it and shows a report a normal developer can understand and fix.

3 Problem Statement

Right now, AI coding tools are really good at writing code, but they don't always check whether that code is secure.

There are already tools that can find security problems, but each tool does a different job. For example, one might check if you accidentally exposed a password or API key, another checks if you're using outdated libraries, and another looks for insecure coding practices.

The problem is that developers have to use all these tools separately and understand different reports from each of them. This can be confusing and time-consuming.

Also, many security issues are found too late, sometimes only after the code has been merged or deployed. By then, the problem could already cause damage.

And even when a security issue is detected, the report can be full of complicated security terms, making it hard for an average developer to understand what went wrong and how to fix it.

Over time, repositories can also become cluttered with old files, unused code, and outdated documentation, making it harder to understand what is actually important.

In short, the pain points are:

- **Too many separate tools:** Developers have to check different types of security issues using different tools.
- **Problems are found too late:** Security mistakes might only be discovered after the code is merged or deployed.
- **Reports are hard to understand:** Developers may not know what the problem means or how to fix it.
- **Repositories become messy:** Old and unused files make the project harder to manage.

4 Proposed Solution

4.1 Overview

The idea is simple: security checks should happen wherever code enters the picture, and every check should end up in one shared place, with one shared way of explaining what's wrong and how to fix it.

There are four modules, each attached to a different moment in a developer's workflow:

ENTRY POINTS

GitHub repo -----> Module 1 (scan the repo, give a fix report)

IDE / editor -----> Module 2 (check code as it is written, before push)

AI prompt -----> Module 3 (make the prompt safer, before code is even generated)

Scheduled scan -----> Module 4 (deep periodic scan + clean-up of junk files/branches)

|
v

- same list of checks, same rule book
- all findings collected in one place, duplicates merged
- ranked by how serious/reachable the problem really is
- every finding explained in plain words + a fix

|
v

OUTPUT

- comment on the pull request (Module 1)
- warning inside the editor (Module 2)
- safer prompt sent to the AI model (Module 3)
- one combined report + dashboard (Module 4)

4.2 What Each Module Actually Does

4.2.1 Module 1 – Repo Scan and Fix Report (GitHub repo)

- Connects to an existing GitHub repository.
- Scans the code for leaked passwords/keys, outdated or risky libraries, and common unsafe code patterns.
- Turns every problem found into a plain-language report, what the risk is, and exactly how to fix it and posts it as a comment on the pull request, so it shows up right where the developer is already looking.
- Never changes code by itself. It only suggests a fix; a person has to accept it.

4.2.2 Module 2 – Editor Check (IDE)

- Works inside the code editor while new code is being written or pasted, checking only the new changes being made instead of scanning the entire existing repository.
- Catches security problems immediately, such as accidentally exposing an API key or password, using an unsafe coding pattern, or introducing other risky code before it is pushed to GitHub.
- Acts as an early warning system for future code, helping developers fix problems as they write them so new security issues don't keep getting added to the repository.

4.2.3 Module 3 – Prompt Hardening (AI prompt)

- Steps in before the code is even generated by an AI coding assistant.
- Quietly adds the safety instructions developers usually forget to mention — things like “use parameterised queries, not string concatenation”, “add rate limiting on login”, “never hardcode secrets, use environment variables”.
- Does not find problems itself, it stops many problems from ever being created in the first place, which makes it the cheapest and highest-value piece to build.

4.2.4 Module 4 – Deep, Scheduled Scan

- Runs on a schedule (nightly or weekly), not on every single push, so it has time to do a slower, deeper pass than Modules 1 and 2.
- Looks for issues that need more time to check properly, and produces one combined report ranked by how serious/reachable each issue actually is.

- Also cleans house: flags files that are no longer used, duplicate or leftover code, and outdated markdown/notes that have piled up in the repo, so the project stays easy to navigate. Like everything else, it only suggests removals for a person to approve, it never deletes anything on its own.

4.3 How the Four Modules Connect Into One Project

All four modules share the same underlying rule book (the same checks for leaked keys, unsafe patterns, and risky libraries), so the logic is written once and reused everywhere, instead of four separate tools that each do their own thing.

Every finding, no matter which module caught it, is written in the same simple format: what file, what line, what the risk is in plain words, and how to fix it.

A shared engine then takes all of this in and does three things: it removes duplicate findings (if two modules catch the same issue, it becomes one entry, not two), it ranks findings by how serious and reachable they really are, and it turns each one into a plain-language explanation with a fix.

This is what makes it one project instead of four disconnected tools: one rule book, one shared brain, four different doors into it.

The order these run in also matters: Module 3 acts before any code exists (prevention), Modules 1 and 2 act as the code is being written and pushed (interception), and Module 4 acts afterwards on a schedule (audit and clean-up).

4.4 Keeping It Practical to Build

- Keep it usable on normal laptops and normal web hosting, no heavy infrastructure needed.
- Reuse well-known, already-trusted tools for the boring, solved parts (see Section 8), and spend original effort only where nothing existing already does the job well.
- Nothing is ever auto-applied without a human accepting it: every fix and every file removal is a suggestion, never an automatic action.

5 Solution Approach – Engineering Focus

5.1 How Detection Works, Kept Simple

Detecting a leaked key or password does not need anything exotic. It comes down to three simple checks used together:

1. Matching against known key formats (an AWS key always starts a certain way, a Slack token always starts a certain way).
2. Checking how random-looking a string is (real secrets look random, English words don't).
3. Flagging risky filenames on sight (like `.env` or `id_rsa`) before even reading what's inside them.

This same detection logic is built once, as a shared piece, and simply called by Module 1 on push and by Module 2 on keystroke — not written twice.

5.2 Basic Architecture

IDE Extension:

Developers install and enable the extension in their IDE. Once enabled, it continuously checks any new code they write or paste and warns them about possible security issues before the code is pushed.

Shared Security Engine:

This is the main brain of the system. It runs the security checks used by the extension, identifies problems such as leaked credentials, unsafe code patterns, and risky dependencies, removes duplicate findings, and ranks issues based on how serious and reachable they are. It then explains each problem in simple language and provides clear suggestions for fixing it.

Storage and History:

Keeps track of previously detected issues, where they were found, and whether they have been fixed or ignored. This prevents the system from repeatedly reporting the same issue and allows developers to see how security problems have changed over time.

For security reasons, it never stores the actual value of a leaked password, API key, or other secret.

5.3 Fast, Safe Delivery

Every change to the tool is automatically built and tested before it goes anywhere, and can be pushed to a staging version safely and often.

This keeps releases small, frequent, and low-risk instead of one big risky release.

6 Evaluation Criterion

6.1 Primary Metric

Time-to-catch: how long between a risky piece of code being written/pushed, and the tool flagging it.

Target: under a couple of minutes for Module 1 and Module 2, since those are the fast, in-the-moment checks.

6.2 Secondary Metrics

- False-positive rate on the checks — kept as close to zero as possible, since noisy alerts get switched off by users.
- Number of real issues found and actually fixed, not just flagged.
- Number of junk files/outdated docs correctly identified by Module 4's clean-up pass.
- A simple 1–5 clarity score from test users — can they understand and act on a report without help.

7 Scalability

In theory, the system grows easily because the four entry points are just different doors into one shared engine — adding a new type of check means teaching the shared engine once, not four times.

The pieces are also kept decoupled from each other (editor extension, GitHub app, scheduled job all talk to the same engine independently), which lets any one of them be scaled up on its own if it gets more usage.

In practice, it can run on ordinary, common hosting: a queue to handle scan jobs one at a time without blocking, short-lived isolated workers to do the actual scanning safely, and a normal database to store the findings history.

8 Engine Availability Heuristic – What Already Exists, So We Don’t Rebuild It

What’s Needed	Existing Tool to Lean On	Build or Reuse
Finding leaked keys/passwords	Gitleaks / TruffleHog (wrapped, with our own allow-list on top)	Mostly reuse
Finding outdated/risky libraries	OSV-Scanner	Reuse
Finding unsafe code patterns	Semgrep, with our own rule set for AI-generated code	Reuse engine, write our own rules
Connecting to GitHub on push	GitHub App + webhook framework (e.g. Probot)	Reuse
Running scan jobs without blocking	A simple job queue (e.g. Redis-based)	Reuse
Editor integration	Standard editor-extension APIs	Build the extension, reuse the rule engine

Using proven tools for the solved parts means the real, original work goes into the parts nobody else does well: the prompt-hardening step, the shared plain-language explanations, the ranking by real-world risk, and the junk-file clean-up pass.

9 Project Scope and Deliverables

9.1 First Working Version

- Module 1: connect to a repo, catch leaked keys and outdated libraries, post a plain-language comment with a fix on the pull request.
- The shared engine: one findings format, duplicate removal, plain-language explanations.
- Basic automatic build + test on every change.

9.2 Later Versions

- Module 2: editor extension, reusing the same rule engine, checking code as it's typed.
- Module 3: the prompt-hardening step, tested by comparing generated code before and after.
- Module 4: the deeper scheduled scan, plus the junk-file/outdated-docs clean-up suggestions and a combined dashboard.

10 Risks and Mitigations

- Too many false alarms could get the tool ignored, fix by shipping the allow-list (test files, placeholders, docs) from day one, not as an afterthought.
- Module 4 wrongly flagging a file as “junk” that’s actually needed, fix by only ever suggesting removal for human approval, never deleting anything automatically.
- Accidentally exposing a secret while scanning for it, fix by never logging or storing the raw secret value, only a masked version.
- Developers not adopting the tool because reports are hard to understand — fix by keeping every single finding in plain words with a clear next step, checked with real users.

11 Summary

This project builds one connected security tool for AI-written code, entered from four different points — the GitHub repo, the code editor, the AI prompt itself, and a scheduled deep scan — but all sharing one rule book and one shared engine underneath.

It focuses on delivering something useful fast, reusing solid existing tools instead of reinventing them, and measuring success with a clear, simple metric: how quickly a real problem gets caught and how easily a normal developer can fix it once it is.